

Satz 1.9 (Chinesischer Restsatz)

Sind $n_1, \dots, n_r$ paarweise teilerfremde natürliche Zahlen, so besitzt für gegebene ganze Zahlen $a_1, \dots, a_r$ das System von Kongruenzen

$$\begin{array}{rcl} x & \equiv & a_1 \pmod{n_1} \\ x & \equiv & a_2 \pmod{n_2} \\ & \vdots & \\ x & \equiv & a_r \pmod{n_r} \end{array}$$

eine eindeutig bestimmte Lösung $a \in \mathbb{Z}$ mit:

$$0 \leq a < n_1 n_2 \cdots n_r$$

Die Menge aller ganzzahligen Lösungen ist durch

$$\mathbb{L} = \{a + v \cdot n_1 n_2 \cdots n_r \mid v \in \mathbb{Z}\}$$

gegeben.

Beweis Zur Konstruktion einer Lösung setzen wir

$$n := n_1 n_2 \cdots n_r$$

und für $i = 1, \dots, r$:

$$m_i := \prod_{j=1, j \neq i}^r n_j = \frac{n}{n_i} = n_1 \cdot n_2 \cdots \cancel{n_i} \cdots n_r$$

EEA liefert u_i, v_i mit:

$$u_i \cdot n_i + v_i \cdot m_i = 1 = \text{ggT}(n_i, m_i)$$

$$e_i := v_i \cdot m_i \quad (i = 1, \dots, r)$$

$e_i \equiv 1$	$(\text{mod } n_i)$
$e_i \equiv 0$	$(\text{mod } m_i)$

$$\tilde{a} := a_1 \cdot e_1 + a_2 \cdot e_2 + \dots + a_r \cdot e_r$$

$$\tilde{a} \text{ mod } n_i = a_i \quad \text{f.ä. } i$$

$$a := \tilde{a} \text{ mod } n$$

$$a \text{ mod } n_i = a_i \quad \text{f.ä. } i$$

$$a_j e_j \text{ mod } n_i = \begin{cases} 0 & \text{für } j \neq i \\ a_j \cdot 1 \text{ mod } n_i, & j = i \end{cases}$$

$$\text{Bemerkung: } n_i \mid m_j \quad \text{f.ä. } j \neq i$$

$$\mathbb{L} = \{a + v \cdot n \mid v \in \mathbb{Z}\}$$

$$\underline{z.z.} \quad |\mathbb{L} \cap \{0, \dots, n-1\}| = 1$$

$$\text{Seien } a, a' \in \mathbb{L} \Rightarrow n_i \mid (a - a') \quad \text{für } i = 1, \dots, r$$

$$n_i \text{ teilerfremd} \Rightarrow n \mid (a - a')$$

Beispiel 1.2

Eine Lösung für folgendes System von Kongruenzen soll berechnet werden:

$$x \equiv 2 = e_1 \pmod{5}$$

$$u_1 = 5$$

$$x \equiv 7 = e_2 \pmod{21}$$

$$u_2 = 21$$

$$x \equiv 6 = e_3 \pmod{11}$$

$$u_3 = 11$$

Musterlösung s. Skript

Unter Übung als Haus Test

$$u_1 = 231, \dots$$

Kartesisches Produkt von Ringen

Sind $R_1, R_2, \dots, R_r$ Ringe, so bildet das kartesische Produkt

$$\mathcal{R} = R_1 \times R_2 \times \dots \times R_r$$

mit den Verknüpfungen

$$(a_1, \dots, a_r) + (b_1, \dots, b_r) = (a_1 + b_1, \dots, a_r + b_r)$$

$$(a_1, \dots, a_r) \cdot (b_1, \dots, b_r) = (a_1 \cdot b_1, \dots, a_r \cdot b_r)$$

ebenfalls einen Ring.

$$0 = (0, 0, \dots, 0)$$

$$1 = (1, 1, \dots, 1)$$

Kommentar Gilt

$$R = R_1 \times R_2 \times \dots \times R_r$$

so gilt für die Einheitengruppen (Menge der invertierbaren Elemente) der Ringe:

$$R^* = R_1^* \times R_2^* \times \dots \times R_r^* \quad (1.7)$$

$$e = (e_1, e_2, \dots, e_r)$$

$$e^{-1} = (e_1^{-1}, e_2^{-1}, \dots, e_r^{-1})$$

Isomorphie von Ringen

Eine bijektive Abbildung zwischen zwei Ringen R_1 und R_2

$$f : R_1 \rightarrow R_2$$

wird *Isomorphismus* genannt, falls für alle $a, b \in R_1$ gilt:

$$f(a + b) = f(a) + f(b)$$

$$f(a \cdot b) = f(a) \cdot f(b)$$

Gibt es einen Isomorphismus zwischen R_1 und R_2 , so heißen die Ringe *isomorph*. In Zeichen:

$$R_1 \cong R_2$$

Satz 1.10

Sind $n_1, n_2, \dots, n_r$ paarweise teilerfremde natürliche Zahlen, so gilt:

$$\mathbb{Z}_n = \mathbb{Z}_{n_1 n_2 \dots n_r} \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \dots \times \mathbb{Z}_{n_r} = \mathbb{R}$$

$$|\mathbb{Z}_n| = n = n_1 \cdot \dots \cdot n_r = |\mathbb{R}|$$

Beweis Die Abbildung

$$f: \mathbb{Z}_{n_1 n_2 \dots n_r} \rightarrow \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \dots \times \mathbb{Z}_{n_r}$$

mit

$$f(a) = (a \bmod n_1, \dots, a \bmod n_r) \quad \text{injektiv} \quad \begin{array}{l} f(a) = f(b) \Rightarrow n_i \mid (a-b) \text{ f.ä. i} \\ \Rightarrow n \mid (a-b) \Rightarrow a-b=0 \Rightarrow a=b \\ |a-b| < n \end{array}$$

ist ein Isomorphismus.

$$f(a+b) = f(a) + f(b) : a, b \in \mathbb{Z}_n, \quad \underline{n = n_1 \cdot n_2 \cdot \dots \cdot n_r}$$

$$\begin{aligned} f(a) + f(b) &= (a \bmod n_1, \dots, a \bmod n_r) + (b \bmod n_1, \dots, b \bmod n_r) \\ &= ((a+b) \bmod n_1, \dots, (a+b) \bmod n_r) \\ &= (((a+b) \bmod n) \bmod n_1, \dots, ((a+b) \bmod n) \bmod n_r) = f(a+b) \end{aligned}$$

$$\text{Es gilt: } f(a) \cdot f(b) = f(a \cdot b) \quad \left| \begin{array}{l} \text{CRT liefert zu } (a_1, \dots, a_r) \in \mathbb{R} \text{ das} \\ \text{Wird } a \in \mathbb{Z}_n \end{array} \right.$$

Der Beweis des chinesischen Restsatzes beinhaltet darüber hinaus ein konstruktives Verfahren zur Berechnung von Urbildern für gegebene Elemente $(a_1, \dots, a_r)$ der Zielmenge von f . Die Elemente $e_1, \dots, e_r$ aus dem Beweis von Satz 1.9 entsprechen dabei den Urbildern von:

$$(1, 0, \dots, 0), (0, 1, 0, \dots, 0), \dots, (0, \dots, 0, 1)$$

Wichtig Ein Isomorphismus bildet invertierbare Elemente auf invertierbare Elemente ab und wegen (1.7) gilt bezüglich der in Satz 1.10 angegebenen Isomorphie

$$\mathbb{Z}_{n_1 \cdot n_2 \dots n_r}^* \cong \mathbb{Z}_{n_1}^* \times \mathbb{Z}_{n_2}^* \times \dots \times \mathbb{Z}_{n_r}^* \quad (1.8)$$

(Die Isomorphie bezüglich der Einheitengruppen bezieht sich natürlich nur auf die multiplikative Verknüpfung.)

Bitte beachten Sie, dass (1.8) zusammen mit den Sätzen 1.7 und 1.8 eine strukturelle Beschreibung der Einheitengruppen $\mathbb{Z}_n^*$ für alle $n \in \mathbb{N}$ liefert.

Insbesondere ergibt sich die bereits im Studienheft Datensicherheit 01, Satz 2.13 behauptete Multiplikativität der Euler'schen ϕ -Funktion für teilerfremde Faktoren.  

$$\phi(n) = |\mathbb{Z}_n^*| \stackrel{(1.8)}{=} \prod_{i=1}^r |\mathbb{Z}_{n_i}^*| = \prod_{i=1}^r |\mathbb{Z}_{p_i^{e_i}}^*| = \prod_{i=1}^r \phi(p_i^{e_i}) = \prod_{i=1}^r p_i^{e_i-1} (p_i - 1)$$

$n = p_1^{e_1} \cdot \dots \cdot p_r^{e_r}$ $\prod_{i=1}^r \phi(n_i)$

Beispiel 1.3 $\mathbb{Z}_{143} \cong \mathbb{Z}_{11} \times \mathbb{Z}_{13}$

Für $n = 143 = 11 \cdot 13$ soll der im Beweis von Satz 1.10 beschriebene Isomorphismus

$$f: \mathbb{Z}_{143} \rightarrow \mathbb{Z}_{11} \times \mathbb{Z}_{13}$$

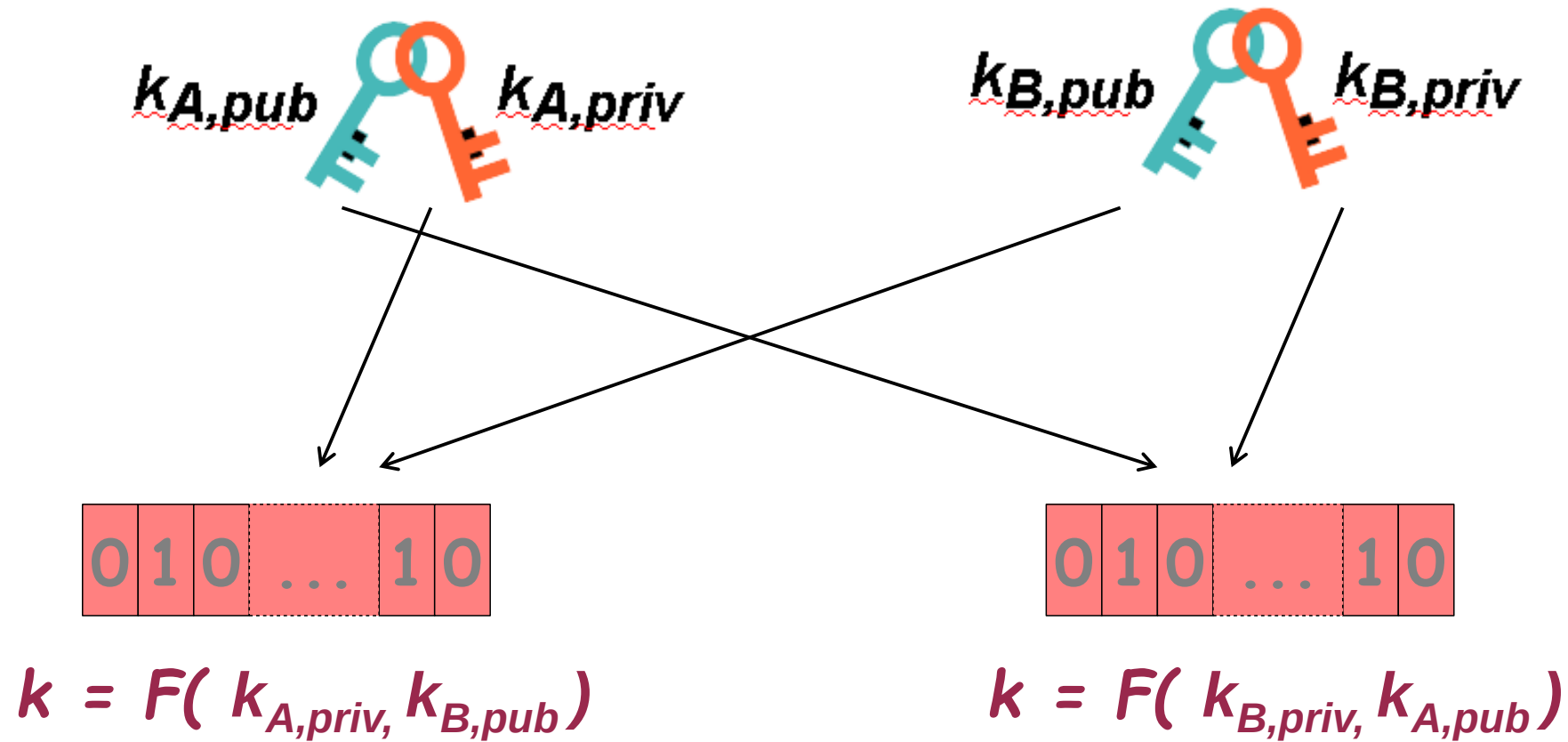
konkret nachvollzogen werden.

Lösung (mit Musterlösung)

–, Hier-Lösung

Das Diffie-Hellman-Schlüsselaustauschverfahren

2.1	Exkurs: Berechnung modularer Potenzen	30
2.2	Exkurs: Angriffe auf das DH-Verfahren	34



Für Schlüsselpaare, die in asymmetrischen kryptographischen Verfahren zum Einsatz kommen, sollten folgende grundlegende Eigenschaften gelten:

- Ein Schlüsselpaar ist genau einem/einer „Besitzer/in“ zugeordnet. Alice besitzt beispielsweise das Schlüsselpaar:

$$k_A = (k_{A,priv}, k_{A,pub})$$

- Niemals hat jemand anderes als Alice Zugriff auf $k_{A,priv}$.



- Konsequenterweise muss Alice $k_{A,priv}$ selbständig generieren.

- Unter Verwendung von $k_{A,priv}$ berechnet Alice auch $k_{A,pub}$ mithilfe eines effizient ausführbaren Algorithmus.

- Es gibt keinen effizient ausführbaren Algorithmus mit dem $k_{A,priv}$ bestimmt werden kann, falls nur $k_{A,pub}$ bekannt ist.

$k_{A,priv} \rightarrow k_{A,pub}$
ist Einwegfunktion
Trap-Door-Funktion

Diffie-Hellman-Verfahren (DH-Verfahren)

- Alice und Bob vereinbaren zunächst zwei Parameter, eine große natürliche Zahl n und eine Zahl:

$$\underline{g \in \mathbb{Z}_n^*}$$

Die Ordnung $o(g)$ von g muss hierbei mindestens durch eine sehr große Primzahl geteilt werden.

Alle weiteren Rechnungen werden in $\mathbb{Z}_n$ ausgeführt.

(Häufig nutzen Alice und Bob DH-Parameter n und g , die in einem Standard speziell zur Nutzung mit dem DH-Verfahren spezifiziert sind.)

- Alice bestimmt eine zufällige Zahl $e_A \in \mathbb{Z}_n^*$ als privaten Schlüssel, d. h.:

$$k_{A,priv} = e_A$$

und berechnet dann den öffentlichen Schlüssel in $\mathbb{Z}_n$:

$$k_{A,pub} = g^{e_A} \in \mathbb{Z}_n$$

Das Schlüsselpaar von Alice ist somit:

$$(k_{A,priv}, k_{A,pub}) = (e_A, g^{e_A})$$

- Bob bestimmt ebenso eine zufällige Zahl $e_B \in \mathbb{Z}_n^*$ und damit das Schlüsselpaar:

$$(k_{B,priv}, k_{B,pub}) = (e_B, g^{e_B})$$

- Alice und Bob tauschen ihre öffentlichen Schlüssel aus, d. h. Alice übermittelt g^{e_A} an Bob und Bob sendet g^{e_B} an Alice.

- Alice berechnet in $\mathbb{Z}_n$:

$$s_A := (g^{e_B})^{e_A}$$

Bob berechnet in $\mathbb{Z}_n$:

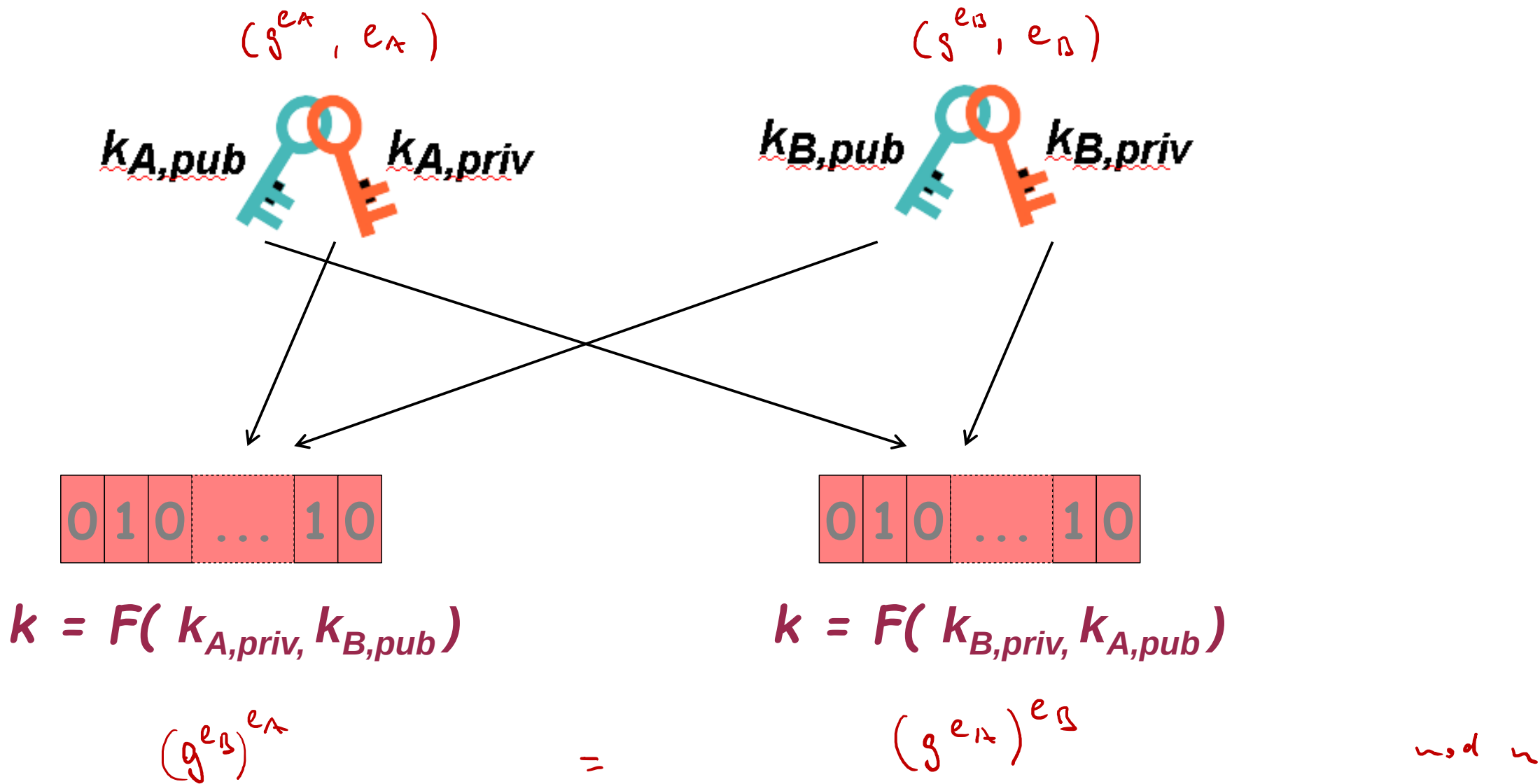
$$s_B := (g^{e_A})^{e_B}$$

$$= g^{e_A \cdot e_B}$$

- Es gilt

$$s_A = (g^{e_B})^{e_A} = g^{e_B \cdot e_A} = g^{e_A \cdot e_B} = (g^{e_A})^{e_B} = s_B$$

und $s_A = s_B$ ist der zwischen Alice und Bob vereinbarte Schlüssel, beziehungsweise ein sogenanntes *shared secret* aus dem weitere Schlüssel zur Verwendung in symmetrischen kryptographischen Verfahren geeignet extrahiert beziehungsweise abgeleitet werden.



Diskrete-Logarithmus-Problem (DL-Problem)

Gegeben seien $n \in \mathbb{N}$, $g \in \mathbb{Z}_n^*$ und eine Potenz von g in $\mathbb{Z}_n$:

$$a \in \langle g \rangle := \{g^e \mid e \in \mathbb{N}\}$$

Die Aufgabenstellung, unter diesen Voraussetzungen einen geeigneten Exponenten e_A mit

$$a = g^{e_A}$$

zu finden, wird als DL-Problem bezeichnet.

Es wird vermutet, dass bei einer geeigneter Wahl von n und g kein Algorithmus existiert, der das DL-Problem effizient löst.

$$e_a = \log_g(a)$$

Diffie-Hellman-Problem (DH-Problem)

Gegeben seien $n \in \mathbb{N}$, $g \in \mathbb{Z}_n^*$ und zwei Potenzen von g in $\mathbb{Z}_n$:

$$g^{e_A}, g^{e_B} \in \langle g \rangle$$

Die Aufgabenstellung, unter diesen Voraussetzungen die Zahl

$$s = g^{e_A \cdot e_B}$$

zu finden, wird als DH-Problem bezeichnet.

Es wird vermutet, dass bei einer Wahl von n und g , für die das DL-Problem nicht effizient lösbar ist, auch kein Algorithmus existiert, der das DH-Problem effizient löst.

Beispiel 2.1 (DH-Parameter)

DH-Parameter sind beispielsweise im RFC 7919 (Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)) im Appendix A zu finden. Die dort angegebenen fünf verschiedenen Zahlen für n sind Primzahlen mit Bitlängen von 2048, 3072, 4096, 6144 und 8192. Die fünf Zahlen sind sogar sogenannte sichere Primzahlen, bei denen auch $\frac{p-1}{2}$ eine Primzahl ist. Solche Primzahlen werden auch als *Sophie-Germain-Primzahlen* bezeichnet.

In allen fünf Fällen ist $g = 2$ und es gilt $o(g) = \frac{p-1}{2}$. Zum Beispiel ist der spezifizierte Wert für n mit einer Länge von 2048 Bitstellen:

$$\begin{aligned} n &= p-1 \\ |\mathbb{Z}_n^*| &= n-1 \\ &= 2 \cdot \frac{n-1}{2} \\ &\quad \uparrow \\ &= p-1 \end{aligned}$$

$n_{2048} =$ 32317006071311007300153513477825163362488057133489
07517458843413926980683413621000279205636264016468
54585563579353308169288290230805734726252735547424
61245741026202527916572972862706300325263428213145
76693141422365422094111134862999165747826803423055
30863490506355577122191878903327295696961297438562
41741236237225197346402691855797767976823014625397
93305801522685873076119753243646747585546071504389
68449403661304976978128542959586595975670512838521
32784468522925504568272879113720098931873959143374
17583782600027803497319855206060753323412260325468
40881200311059074842810039949669561196969562486290
32338072839127039

Zu beachten ist, dass der Aufwand eines Brute-Force-Angriffs auf das DL-Problem nicht direkt mit der Bitlänge von n abzuschätzen ist. In RFC 7919 wird beispielsweise die Größe des effektiv zu durchsuchenden Raumes als „*estimated symmetric-equivalent strength*“ bezeichnet und für $n = n_{2048}$ mit 2^{103} abgeschätzt. ◀

security strength

Die Frage, ob es überhaupt einen Primzahltest gibt, dessen Laufzeit polynomial durch die Stellenzahl (z. B. Bitlänge) der zu überprüfenden Zahl beschränkt ist, war lange Zeit offen. Im Jahr 2002 konnte sie von M. Agrawal, N. Kayal und N. Saxena in ihrer Arbeit PRIMES is in P positiv entschieden werden. Der entsprechende Primzahltest heißt *AKS-Test*.

In der Praxis kommen daher sogenannten *probabilistische Primzahltest*, die mit einer Fehlerwahrscheinlichkeit p parametrisiert werden können, zum Einsatz. Für jede natürliche Zahl n liefert ein probabilistischer Primzahltest eine der beiden Aussagen zurück:

- n ist keine Primzahl.

Dann ist n definitiv keine Primzahl. Der Algorithmus liefert aber in der Regel keinen echten Teiler von n .

- n ist wahrscheinlich eine Primzahl.

(kl. Satz von Fermat)

$$p \text{ Primzahl} \Rightarrow \mathbb{Z}_p^* \text{ zyklisch} \Rightarrow \forall a \in \mathbb{Z}_p^* : a^{p-1} \equiv 1 \pmod{p}$$

$$a^{p-1} \pmod{p} = 1 \neq 1 \Rightarrow p \text{ keine Primzahl}$$

In der Praxis kommen daher sogenannten *probabilistische Primzahltest*, die mit einer Fehlerwahrscheinlichkeit p parametrisiert werden können, zum Einsatz. Für jede natürliche Zahl n liefert ein probabilistischer Primzahltest eine der beiden Aussagen zurück:

- n ist keine Primzahl.

Dann ist n definitiv keine Primzahl. Der Algorithmus liefert aber in der Regel keinen echten Teiler von n .

- n ist wahrscheinlich eine Primzahl.

(kl. Satz von Fermat)

$$p \text{ Pz} \Rightarrow \mathbb{Z}_p^* \text{ zyklisch} \Rightarrow \forall a \in \mathbb{Z}_p^* \quad a^{p-1} \equiv 1 \pmod{p}$$

$$a^{p-1} \pmod{p} = 1 \neq 1 \Rightarrow p \text{ keine Pz}$$

Frage: n keine Pz, $\left| \{a \in \mathbb{Z}_n \mid a^{n-1} \pmod{n} = 1\} \right| / (n-1) \left[\begin{smallmatrix} \approx 0 \\ ? \end{smallmatrix} \right]$

Gibt keine spezielle Zahlen, wo der Test nicht ausreicht ist (Carmichael - Zahlen)

Miller-Rabin-Pz-Test: $p-1 = 2^e \cdot n$: $a \mapsto a^{2^e \cdot n} = 1 \pmod{p}$ ($\neq 1 \rightarrow$ keine Pz)

$$b = a^{2^{e-1} \cdot n} = \frac{1}{-1} \pmod{p} \quad (\neq 1, -1 \rightarrow \text{keine Pz})$$

$$b^2 = a^{2^e \cdot n} = 1 \quad b = a^n = 1, -1$$

2.2.1 Pohlig-Hellman-Reduktion

2.2.2 Shanks' „Baby Steps Giant Steps“-Verfahren (BSGS-Verfahren)

2.2.3 Pollard's *Rho-Methode*

2.2.4 Zahlkörpersieb

Es sei n eine Primzahl

CRT

$w_i = p_i^{e_i}$

$$n - 1 = p_1^{e_1} \cdots p_r^{e_r} \quad m_i := \frac{n - 1}{p_i^{e_i}} \quad \text{für } i = 1, \dots, r$$

$$v_i \in \mathbb{Z}_{p_i^{e_i}} \text{ mit: } v_i \cdot m_i \bmod p_i^{e_i} = 1 \quad \text{für } i = 1, \dots, r$$

$$\tilde{a} = \sum_{i=1}^r v_i \cdot m_i \quad \tilde{a} \equiv 1 \pmod{p_i^{e_i}} \quad \text{für } i = 1, \dots, r$$

$$\left(\sum_{i=1}^r v_i \cdot m_i \right) = \tilde{a} \equiv 1 \pmod{n - 1} \quad (*)$$

$$g^x = a \quad o(g) = n - 1$$

$$g_i := g^{m_i} \quad a_i := a^{m_i}$$

$$g_i^{x_i} = a_i \quad \text{für } i = 1, \dots, r$$

mit: $x_i \in \mathbb{Z}_{p_i^{e_i}}$

$$x := \left(\sum_{i=1}^r v_i \cdot m_i \cdot x_i \right) \bmod (n - 1)$$

$$g^x = g^{\sum_{i=1}^r v_i \cdot m_i \cdot x_i} = \prod_{i=1}^r g^{v_i \cdot m_i \cdot x_i} = \prod_{i=1}^r g_i^{v_i \cdot x_i} = \prod_{i=1}^r a_i^{v_i} = \prod_{i=1}^r a^{m_i \cdot v_i} \stackrel{(*)}{=} a$$

2.2.2 Shanks' „Baby Steps Giant Steps“-Verfahren (BSGS-Verfahren)

Sei $g \in \mathbb{Z}_n^*$ und $m = o(g)$.

$$g^x = a$$

Meet in the middle Aufwand $\sim \sqrt{n}$
Speicher $\sim \sqrt{n}$

$$\tilde{m} := \lceil \sqrt{m} \rceil$$

$$x = x_2 \cdot \tilde{m} + x_1 \quad \text{mit } x_1, x_2 \in \{0, \dots, \tilde{m} - 1\}$$

$$g^x = a \iff g^{\tilde{m} \cdot x_2} \cdot g^{x_1} = a \iff g^{x_1} = a \cdot (g^{-\tilde{m}})^{x_2}$$

$$g^{x_i} \cdot a^{y_i}$$

$$(g, a)^{(x, y)} := g^x \cdot a^y \quad T_i = (x_i, y_i, (g, a)^{(x_i, y_i)})$$

$$g^{x_i} \cdot a^{y_i} = g^{x_j} \cdot a^{y_j} \iff g^{x_i - x_j} = a^{y_j - y_i}$$

$$f : \mathbb{Z}_m^2 \rightarrow \mathbb{Z}_m^2 \text{ mit: } (g, a)^{(x, y)} = (g, a)^{(\tilde{x}, \tilde{y})} \implies (g, a)^{f(x, y)} = (g, a)^{f(\tilde{x}, \tilde{y})}$$

$$(x_2, y_2) = f(x_1, y_1), \quad (x_3, y_3) = f(x_2, y_2) = f^2(x_1, y_1), \quad \dots$$

Zu Beginn der 1990er Jahre wurde das *Zahlkörpersieb* (*Number Field Sieve* (NFS)) als Methode zur Faktorisierung großer Zahlen n und zur Bestimmung diskreter Logarithmen in $\mathbb{Z}_n^*$ entwickelt. Die Laufzeit zur Bestimmung diskreter Logarithmen in $\mathbb{Z}_n^*$ mithilfe des Zahlkörpersiebs kann durch

$$O\left(e\left(\left(\frac{64}{9}\right)^{1/3} + o(1)\right)(\ln(n))^{1/3}(\ln(\ln(n)))^{2/3}\right)$$

von oben abgeschätzt werden.

Elliptische Kurven können über beliebigen Körpern K definiert werden.

In kryptographischen Anwendungen ist K meist ein endlicher Körper von Primzahlordnung $K = \mathbb{F}_p$ oder ein Körper der Charakteristik 2, d. h. $K = \mathbb{F}_{2^n}$.

Elliptische Kurven über $K = \mathbb{F}_p$ (Weierstraß-Form)

Sei $p > 3$ eine Primzahl seien $a, b \in \mathbb{F}_p$ mit:

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

Dann definiert

$$y^2 = x^3 + ax + b$$

eine *elliptische Kurve* über $\mathbb{F}_p$, bestehend aus folgenden *Punkten*:

$$E(\mathbb{F}_p) := \{(x, y) \mid x, y \in \mathbb{F}_p, y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

$\mathcal{O}$ wird als *Punkt im Unendlichen* bezeichnet.

Addition auf $E(\mathbb{F}_p)$

Eine Addition von Punkten der elliptischen Kurve $E(\mathbb{F}_p)$ wird wie folgt definiert:

(i) Für alle $P \in E(\mathbb{F}_p)$ setze:

$$P + \mathcal{O} := \mathcal{O} + P := P$$

Neutralement

(ii) Für $P = (x, y) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ setze:

$$(x, y) + (x, -y) = \mathcal{O}$$

(iii) Für $P_1 = (x_1, y_1), P_2 = (x_2, y_2) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ mit $x_1 \neq x_2$ setze:

$$s := \frac{y_2 - y_1}{x_2 - x_1}$$

$$x := s^2 - x_1 - x_2$$

$$y := s(x_1 - x) - y_1$$

$$P_1 + P_2 = (x_1, y_1) + (x_2, y_2) := (x, y)$$

Addition auf $E(\mathbb{F}_p)$

Eine Addition von Punkten der elliptischen Kurve $E(\mathbb{F}_p)$ wird wie folgt definiert:

(iv) Für $P = (x_1, y_1) \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$ mit $y_1 \neq 0$ setze:

$$s := \frac{3x_1^2 + a}{2y_1}$$

$$x := s^2 - 2x_1$$

$$y := s(x_1 - x) - y_1$$

$$2 \cdot P_1 = P_1 + P_1 = (x_1, y_1) + (x_1, y_1) := (x, y)$$

Satz

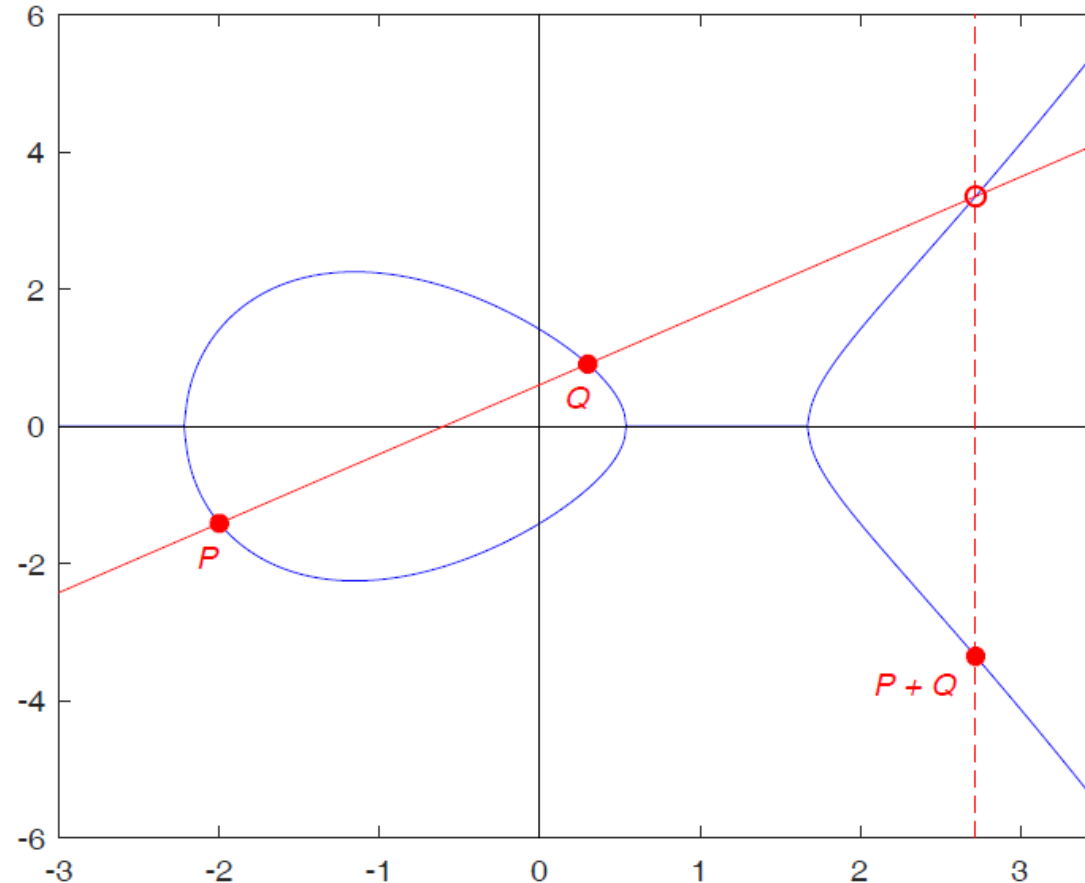
Bezüglich der definierten Addition ist $E(\mathbb{F}_p)$ eine kommutative Gruppe mit Nullelement $\mathcal{O}$ und:

$$-P = \begin{cases} (x, -y) & \text{für } P = (x, y) \\ \mathcal{O} & \text{für } P = \mathcal{O} \end{cases}$$

Kommentar

Ist $E = E(\mathbb{R})$ eine elliptische Kurve über den reellen Zahlen, so lässt sich die Addition von zwei Punkten $P, Q \in E \setminus \{\mathcal{O}\}$ geometrisch interpretieren:

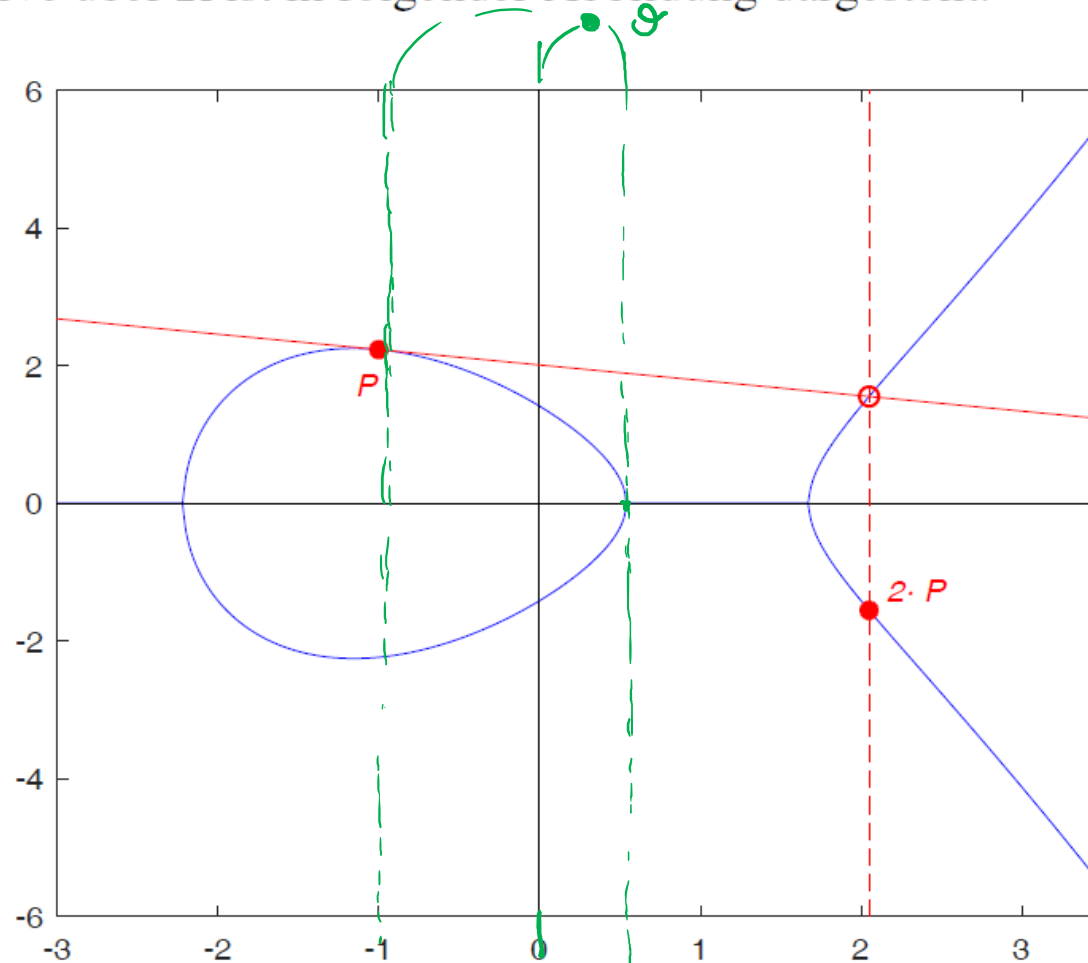
- Für $P \neq \pm Q$ schneidet die Gerade durch P und Q die elliptische Kurve in genau einem weiteren Punkt. Das Negative dieses Punkts (Spiegelung an der x -Achse) ist $P + Q$. Ein Beispiel für die Addition von zwei Punkten der durch $y^2 = x^3 - 4x + 2$ definierten elliptischen Kurve über $\mathbb{R}$ ist in folgender Abbildung dargestellt.



Kommentar

Ist $E = E(\mathbb{R})$ eine elliptische Kurve über den reellen Zahlen, so lässt sich die Addition von zwei Punkten $P, Q \in E \setminus \{\mathcal{O}\}$ geometrisch interpretieren:

- Für $P = Q$ ist die Gerade durch P und Q durch die Tangente durch P zu ersetzen. Ein Beispiel zur Konstruktion des Doppelten eines Punktes der durch $y^2 = x^3 - 4x + 2$ definierten elliptischen Kurve über $\mathbb{R}$ ist in folgender Abbildung dargestellt.



Satz von Hasse

Ist $E(\mathbb{F}_q)$ eine elliptische Kurve über einem endlichen Körper $E(\mathbb{F}_q)$, so gilt:

$$\left| |E(\mathbb{F}_q)| - (q + 1) \right| \leq 2\sqrt{q}$$

$$q = p^n$$

Gruppenstruktur von $E(\mathbb{F}_q)$

Entweder ist $E(\mathbb{F}_q)$ zyklisch oder $E(\mathbb{F}_q) \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2}$ mit $n_1 \mid n_2$.

Diffie-Hellman-Verfahren (DH-Verfahren)

- Alice und Bob vereinbaren zunächst zwei Parameter, eine große natürliche Zahl n und eine Zahl:

$$g \in \mathbb{Z}_n^*$$

Die Ordnung $o(g)$ von g muss hierbei mindestens durch eine sehr große Primzahl geteilt werden.

(Häufig nutzen Alice und Bob DH-Parameter n und g , die in einem Standard speziell zur Nutzung mit dem DH-Verfahren spezifiziert sind.)

$$\mathbb{Z}_n^*, g \in \mathbb{Z}_n^*$$

$$m = o(g) \quad g^{m|3}$$

$$g^m = 1$$

Elliptic-Curve-Diffie-Hellman-Verfahren (ECDH-Verfahren)

- Alice und Bob vereinbaren zunächst eine elliptische Kurve $E(\mathbb{F}_p)$ und einen festen Punkt:

$$G \in E(\mathbb{F}_p)$$

Die Ordnung $o(G)$ von G muss hierbei mindestens durch eine sehr große Primzahl geteilt werden.

(Häufig verwenden Alice und Bob eine Kurve $E(\mathbb{F}_p)$ und einen Punkt $G \in E(\mathbb{F}_p)$, die in einem Standard zur Nutzung mit dem ECDH-Verfahren spezifiziert sind.)

$$E(\mathbb{F}_p), G \in E(\mathbb{F}_p)$$

$$n = o(G) \quad g^{m|3}$$

$$m \cdot G = \underbrace{G + G + \dots + G}_m = 0$$

(on minimal)

- Alice bestimmt eine zufällige Zahl $e_A \in \mathbb{Z}_n^*$ als privaten Schlüssel, d. h.:

$$k_{A,priv} = e_A$$

und berechnet dann den öffentlichen Schlüssel in $\mathbb{Z}_n$:

$$k_{A,pub} = g^{e_A} \in \mathbb{Z}_n$$

Das Schlüsselpaar von Alice ist somit:

$$(k_{A,priv}, k_{A,pub}) = (e_A, g^{e_A})$$

- Alice bestimmt eine zufällige natürliche Zahl $e_A < o(G)$ als privaten Schlüssel, d. h.:

$$k_{A,priv} = e_A$$

und berechnet dann den öffentlichen Schlüssel in $E(\mathbb{F}_p)$:

$$k_{A,pub} = e_A \cdot G \in E(\mathbb{F}_p)$$

Das Schlüsselpaar von Alice ist somit:

$$(k_{A,priv}, k_{A,pub}) = (e_A, e_A \cdot G)$$

- Bob bestimmt ebenso eine zufällige Zahl $e_B \in \mathbb{Z}_n^*$ und damit das Schlüsselpaar:

$$(k_{B,priv}, k_{B,pub}) = (e_B, g^{e_B})$$

- Alice und Bob tauschen ihre öffentlichen Schlüssel aus, d. h. Alice übermittelt g^{e_A} an Bob und Bob sendet g^{e_B} an Alice.

- Alice berechnet in $\mathbb{Z}_n$: $s_A := (g^{e_B})^{e_A}$

Bob berechnet in $\mathbb{Z}_n$: $s_B := (g^{e_A})^{e_B}$

- Bob bestimmt ebenso eine zufällige natürliche Zahl $e_B < o(G)$ und damit sein Schlüsselpaar:

$$(k_{B,priv}, k_{B,pub}) = (e_B, e_B \cdot G)$$

- Alice und Bob tauschen ihre öffentlichen Schlüssel aus, d. h. Alice übermittelt $e_A \cdot G$ an Bob und Bob sendet $e_B \cdot G$ an Alice.

- Alice berechnet in $E(\mathbb{F}_p)$:

$$s_A := e_A \cdot (e_B \cdot G)$$

Bob berechnet in $E(\mathbb{F}_p)$:

$$s_B := e_B \cdot (e_A \cdot G)$$

■ Es gilt

$$s_A = (g^{e_B})^{e_A} = g^{e_B \cdot e_A} = g^{e_A \cdot e_B} = (g^{e_A})^{e_B} = s_B$$

und $s_A = s_B$ ist der zwischen Alice und Bob vereinbarte Schlüssel, beziehungsweise ein sogenanntes *shared secret* aus dem weitere Schlüssel zur Verwendung in symmetrischen kryptographischen Verfahren geeignet extrahiert beziehungsweise abgeleitet werden.

■ Es gilt

$$S_A := e_A \cdot (e_B \cdot G) = (e_A \cdot e_B) \cdot G = (e_B \cdot e_A) \cdot G = e_B \cdot (e_A \cdot G) = S_B$$

und $S_A = S_B = (x_S, y_S)$, bzw. der Wert der x -Koordinate x_S ist der so zwischen Alice und Bob vereinbarte Schlüssel (shared secret).

secp256r1 (NIST SP 800-186: P-256)

$$\begin{aligned}
 p &= 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1 \\
 &= 115792089210356248762697446949407573530 \\
 &\quad 086143415290314195533631308867097853951
 \end{aligned}$$

$$a = p - 3$$

$$\begin{aligned}
 b &= 41058363725152142129326129780047268409 \\
 &\quad 114441015993725554835256314039467401291
 \end{aligned}$$

$$G = (x_G, y_G)$$

$$\begin{aligned}
 x_G &= 48439561293906451759052585252797914202 \\
 &\quad 762949526041747995844080717082404635286
 \end{aligned}$$

$$\begin{aligned}
 y_G &= 36134250956749795798585127919587881956 \\
 &\quad 611106672985015071877198253568414405109
 \end{aligned}$$

dc_260618.pcapng

Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telephonie Wireless Tools Hilfe

_ws.col.protocol == "TLSv1.3"

No.	Time	Source	Source Port	Destination	Protocol	Length	Info
455	35.000	192.168.11.100	56713	193.174.116.151	TLSv1.3	1940	Client Hello (SNI=www.th-owl.de)
459	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1304	Server Hello, Change Cipher Spec, Encrypted Extensions
463	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1200	Certificate, Certificate Verify, Finished
467	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1304	Server Hello, Change Cipher Spec, Encrypted Extensions
471	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1200	Certificate, Certificate Verify, Finished
473	35.000	192.168.11.100	56714	193.174.116.151	TLSv1.3	1940	Client Hello (SNI=www.th-owl.de)

- Extension: key_share (len=1327) X25519MLKEM768, x25519, secp256r1
 - Type: key_share (51)
 - Length: 1327
 - Key Share extension
 - Client Key Share Length: 1325
 - Key Share Entry: Group: X25519MLKEM768, Key Exchange length: 1216
 - Group: X25519MLKEM768 (4588)
 - Key Exchange Length: 1216
 - Key Exchange [...]: 53d8a6dd76ca9347825f1843fcb7777b9c86e3ca65ccf6bffd31e8af3955bc0448dfc7582ca
 - Key Share Entry: Group: x25519, Key Exchange length: 32
 - Group: x25519 (29)
 - Key Exchange Length: 32
 - Key Exchange: ed09af224f98f392c4f194a8ee1b0f93dd98d1ab8d9643d1245866c2a66c9f65
 - Key Share Entry: Group: secp256r1, Key Exchange length: 65
 - Group: secp256r1 (23)
 - Key Exchange Length: 65
 - Key Exchange: 04e847aa7dd5d23e85658d318ca1ba5eae558fbf196dfacbccf136a9411044ad3c664c14927ffa99

```

0100 00 08 04 03 05 03 06 03 02 03
0110 05 2f 05 2d 11 ec 04 c0 53 d8
0120 82 5f 18 43 fc b7 77 7b 9c 86
0130 fd c3 1e 8a f3 95 5b c0 44 8d
0140 41 25 85 97 a0 c5 07 77 c0 51
0150 9c 03 09 a0 03 b7 69 13 73 31
0160 65 fa ac 04 b4 8c bb a0 6d 65
0170 5a cb ea b1 45 c9 4b 73 11 b0
0180 55 05 78 81 af 31 48 85 79 91
0190 f7 ec 96 f0 c4 92 19 91 a6 2a
01a0 f1 1d a2 b0 c9 ff 14 27 ce eb
01b0 62 94 a9 5f df 75 3c 66 30 7a
01c0 62 b0 9b 68 00 c2 f7 e6 00 35
01d0 c4 cc 84 25 2d e6 2c a2 76 dc
01e0 5b b1 e5 72 a0 68 a0 7a 56 45
01f0 e6 4a 6d 89 04 19 d7 5a c9 40
0200 d5 74 28 24 5a a8 06 1b 44 48
0210 c6 f7 e9 9a ce 68 23 08 1a 2c
0220 42 13 33 0e 90 ba dc 10 c6 74
0230 45 74 64 b5 66 42 c7 c5 c0 25
          
```

Hello extension type (tls.handshake.extension.type), 2 Bytes

Datensicherheit – Symmetrische Verfahren

Pakete: 2541 · Angezeigt: 192 (7.6%)

Profil: Default

dc_260618.pcapng

Datei Bearbeiten Ansicht Navigation Aufzeichnen Analyse Statistiken Telephonie Wireless Tools Hilfe

_ws.col.protocol == "TLSv1.3"

No.	Time	Source	Source Port	Destination	Protocol	Length	Info
455	35.000	192.168.11.100	56713	193.174.116.151	TLSv1.3	1940	Client Hello (SNI=www.th-owl.de)
459	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1304	Server Hello, Change Cipher Spec, Encrypted Extensions
463	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1200	Certificate, Certificate Verify, Finished
467	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1304	Server Hello, Change Cipher Spec, Encrypted Extensions
471	35.000	193.174.116.151	443	192.168.114.56	TLSv1.3	1200	Certificate, Certificate Verify, Finished
473	35.000	192.168.11.100	56714	193.174.116.151	TLSv1.3	1940	Client Hello (SNI=www.th-owl.de)
476	35.000	192.168.11.100	56715	193.174.116.151	TLSv1.3	1940	Client Hello (SNI=www.th-owl.de)

Extensions Length: 79

- Extension: key_share (len=69) secp256r1
 - Type: key_share (51)
 - Length: 69
 - Key Share extension
 - Key Share Entry: Group: secp256r1, Key Exchange length: 65
 - Group: secp256r1 (23)
 - Key Exchange Length: 65
 - Key Exchange: 046ba57dbc25cbdba43680ccd41402e05ad835630a3dc29c2f8ec280b09077844bbfe94c735b3db6
- Extension: supported_versions (len=2) TLS 1.3
 - Type: supported_versions (43)
 - Length: 2
 - Supported Version: TLS 1.3 (0x0304)
 - [JA3S Fullstring: 771,4865,51-43]
 - [JA3S: eb1d94daa7e0344597e756a1fb6e7054]

Packet (1304 bytes) Decrypted TLS (6 bytes)

0080 59 91 13 01 00 00 4f 00 33 00

0090 6b a5 7d bc 25 cb db a4 36 80

00a0 d8 35 63 0a 3d c2 9c 2f 8e c2

00b0 bf e9 4c 73 5b 3d b6 ab b1 12

00c0 ae 3f dd 85 35 44 6d 93 9a 27

00d0 00 2b 00 02 03 04 14 03 03 00

00e0 17 72 93 91 e4 f6 03 ff a1 41

00f0 d7 4d b2 e0 91 be f1 7c 17 03

0100 19 5b bb fb 78 a5 d8 0b 96 4f

0110 16 f0 aa 6f 86 1c 1f 53 78 ea

0120 f6 4c 25 5d 8b 27 e1 c8 a3 77

0130 a7 5b ba 9f d8 fe b7 dc 81 c1

0140 89 90 72 d1 fb f3 74 1c 50 29

0150 bd 33 ba 57 75 20 9a e7 fa 82

0160 d1 43 d4 de be 0c 16 3b da f0

0170 8e cd c5 a3 0b 98 80 bb 25 3a

0180 79 b9 38 16 75 0a dd 35 79 0e

Group (tls.handshake.extensions_key_share_group), 2 Bytes

Pakete: 2541 · Angezeigt: 192 (7.6%) Profil: Default

```
18 public class KeyAgreement_Demo {
19
20     public static void main(String[] args) throws NoSuchAlgorithmException,
21         InvalidKeyException, InvalidKeySpecException,
22         InvalidAlgorithmParameterException {
23
24         System.out.println("\n(1) Key-Agreement mit DH (2048 Bit)");
25         KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("DH");
26         keyPairGen.initialize(2048);
27         keyAgreementDemo(keyPairGen, "DH", "DH");
28
29         System.out.println("\n(2) Key-Agreement mit ECDH (secp256r1)");
30         keyPairGen = KeyPairGenerator.getInstance("EC");
31         keyPairGen.initialize(new ECGenParameterSpec("secp256r1"));
32         keyAgreementDemo(keyPairGen, "ECDH", "EC");
33     }
34
35     public static void keyAgreementDemo(KeyPairGenerator keyPairGen,
36         String algoKeyAgreement, String algoKeyFactory) throws
37         NoSuchAlgorithmException, InvalidKeyException, InvalidKeySpecException {
38
39         KeyAgreement keyAgreement = KeyAgreement.getInstance(algoKeyAgreement);
```

```
38
39 KeyAgreement keyAgreement = KeyAgreement.getInstance(algoKeyAgreement);
40 KeyFactory keyFactory = KeyFactory.getInstance(algoKeyFactory);
41
42 // Alice -----
43 KeyPair keyPair_Alice = keyPairGen.generateKeyPair();
44 byte[] pubKeyEnc_Alice = keyPair_Alice.getPublic().getEncoded();
45
46 // Bob -----
47 KeyPair keyPair_Bob = keyPairGen.generateKeyPair();
48
49 keyAgreement.init(keyPair_Bob.getPrivate());
50
51 PublicKey pubKey_Alice = keyFactory.generatePublic(
52     new X509EncodedKeySpec(pubKeyEnc_Alice));
53 keyAgreement.doPhase(pubKey_Alice, true);
54
55 byte[] secret_Bob = keyAgreement.generateSecret();
56 System.out.println("Bob's Shared Secret");
57 System.out.println(Dump.dump(secret_Bob));
58
59 byte[] pubKeyEnc_Bob = keyPair_Bob.getPublic().getEncoded();
60
61 // Alice -----
```